[CHAPTER] 3

Control Statements

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

control statements—statements that allow the computer to select or repeat an action.

We begin our study of control statements with repetition statements, also known as **loops**, which repeat an action. Each repetition of the action is known as a **pass** or an **iteration**. There are two types of loops—those that repeat an action a predefined number of times (**definite iteration**) and those that perform the action until the program determines that it needs to stop (**indefinite iteration**).

The for Loop

Here is a **for** loop that runs the same output statement four times:

```
for <variable> in range(<an integer expression>):
    <statement-1>
    .
    .
    <statement-n>
```

```
>>> for eachPass in range(4):
        print("It's alive!", end = " ")
It's alive! It's alive! It's alive! It's alive!
```

This loop repeatedly calls one function—the **print** function. The constant 4 on the first line tells the loop how many times to call this function. If we want to print 10 or 100 exclamations, we just change the 4 to 10 or to 100.

EIU
EASTERN INTERNATIONAL UNIVERSITY

The first line of code in a loop is sometimes called the **loop header**. The colon (:) ends the loop header. The **loop body** comprises the statements in the remaining lines of code, below the header. These statements are executed in sequence on each pass through the loop. Note that the statements in the loop body *must be indented and aligned in the same column.*

```
for <variable> in range(<an integer expression>):
    <statement-1>
    .
    .
    <statement-n>
```

```
>>> number = 2
>>> exponent = 3
>>> product = 1
>>> for eachPass in range(exponent):
        product = product * number
        print(product, end = " ")
2 4 8
>>> product
8
```

Count-Controlled Loops

```
>>> for count in range(4):
        print(count, end = " ")
0 1 2 3
```

```
>>> product = 1
>>> for count in range(4):
        product = product * (count + 1)
>>> product
24
```

Loops that count through a range of numbers are also called **count-controlled loops**.

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> product = 1
>>> for count in range(1, 5):
        product = product * count
>>> product
24
```

Augmented Assignment

`x = x + 1` or `x = x + 2`

The assignment symbol can be combined with the arithmetic and concatenation operators to provide **augmented assignment operations**.

All these examples have the format

`<variable> <operator>= <expression>`

which is equivalent to

`<variable> = <variable> <operator> <expression>`

```
for <variable> in range(<lower bound>, <upper bound + 1>):
    <loop body>
```

```
>>> lower = int(input("Enter the lower bound: "))
Enter the lower bound: 1
>>> upper = int(input("Enter the upper bound: "))
Enter the upper bound: 10
>>> theSum = 0
>>> for number in range(lower, upper + 1):
        theSum = theSum + number
>>> theSum
55
```

```
a = 17
s = "hi"
a += 3          # Equivalent to a = a + 3
a -= 3          # Equivalent to a = a - 3
a *= 3          # Equivalent to a = a * 3
a /= 3          # Equivalent to a = a / 3
a %= 3          # Equivalent to a = a % 3
s += " there"   # Equivalent to s = s + " there"
```

Loop Errors: Off-by-One Error

Because this number is typically off by one, the error is called an **off-by-one error**.

```
# Count from 1 through 4, we think
>>> for count in range(1,4):
    print(count)
1
2
3
```

Traversing the Contents of a Data Sequence

```
>>> list(range(4))
[0, 1, 2, 3]
>>> list(range(1, 5))
[1, 2, 3, 4]
```

Although we have been using the **for** loop as a simple count-controlled loop, the loop itself visits each number in a sequence of numbers generated by the **range** function. The next code segment shows what these sequences look like:

In this example, the sequence of numbers generated by the function **range** is fed to Python's **list** function, which returns a special type of sequence called a **list**.

```
for <variable> in <sequence>:
    <do something with variable>
```

```
>>> for number in [6, 4, 8]:
        print(number, end = " ")
6 4 8
```

```
>>> for character in "Hi there!":
        print(character, end = " ")
H i  t h e r e !
```

Specifying the Steps in the Range

```
>>> list(range(1, 6, 1))   # Same as using two arguments
[1, 2, 3, 4, 5]
>>> list(range(1, 6, 2))   # Use every other number
[1, 3, 5]
>>> list(range(1, 6, 3))   # Use every third number
[1, 4]
```

```
>>> theSum = 0
>>> for count in range(2, 11, 2):
        theSum += count
>>> theSum
30
```

Loops That Count Down

a loop displays the count from 10 down to 1

```
>>> for count in range(10, 0, -1):
        print(count, end = " ")
10 9 8 7 6 5 4 3 2 1
>>> list(range(10, 0, -1))
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Write the outputs of the following loops:

   a.
   ```
   for count in range(5):
       print(count + 1, end = " ")
   ```

   b.
   ```
   for count in range(1, 4):
       print(count, end = " ")
   ```

   c.
   ```
   for count in range(1, 6, 2):
       print(count, end = " ")
   ```

   d.
   ```
   for count in range(6, 1, -1):
       print(count, end = " ")
   ```

2. Write a loop that prints your name 100 times. Each output should begin on a new line.

3. Assume that the variable **`teststring`** refers to a string. Write a loop that prints each character in this string, followed by its ASCII value.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Formatting Text for Output

The total number of data characters and additional spaces for a given datum in a formatted string is called its **field width**.

```
>>> for exponent in range(7, 11):
        print(exponent, 10 ** exponent)
7 10000000
8 100000000
9 1000000000
10 10000000000
```

```
>>> "%6s" % "four"    # Right justify
'  four'
>>> "%-6s" % "four"   # Left justify
'four  '
```

The first line of code right-justifies the string by padding it with two spaces to its left. The next line of code left-justifies by placing two spaces to the string's right.

```
<format string> % <datum>

<format string> % (<datum-1>, ..., <datum-n>)
```

This version contains a **format string**, the **format operator** `%`, and a single data value to be formatted. The `%` operator works with this information to build and return a formatted string.

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> for exponent in range(7, 11):
        print("%-3d%12d" % (exponent, 10 ** exponent))
7       10000000
8      100000000
9     1000000000
10   10000000000
```

The format information for a data value of type **float** has the form
%<field width>.<precision>f where **.<precision>** is optional.

```
>>> salary = 100.00
>>> print("Your salary is $" + str(salary))
Your salary is $100.0
>>> print("Your salary is $%0.2f" % salary)
Your salary is $100.00
```

```
>>> "%6.3f" % 3.14
' 3.140'
```

Note that Python adds a digit of precision to the string and pads it with a space to the left to achieve the field width of 6. This width includes the place occupied by the decimal point.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises

1. Assume that the variable **amount** refers to 24.325. Write the outputs of the following statements:

   a. `print("Your salary is $%0.2f" % amount)`

   b. `print("The area is %0.1f" % amount)`

   c. `print("%7f" % amount)`

2. Write a code segment that displays the values of the integers **x**, **y**, and **z** on a single line, such that each value is right-justified with a field width of 6.

3. Write a format operation that builds a string for the **float** variable **amount** that has exactly two digits of precision and a field width of zero.

4. Write a loop that outputs the numbers in a list named **salaries**. The outputs should be formatted in a column that is right-justified, with a field width of 12 and a precision of 2.



EIU
EASTERN INTERNATIONAL UNIVERSITY

The Boolean Type, Comparisons, and Boolean Expressions

Table 3-2 The comparison operators

Comparison Operator	Meaning
==	Equals
!=	Not equals
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

```
>>> 4 == 4
True
>>> 4 != 4
False
>>> 4 < 5
True
>>> 4 >= 3
True
>>> "A" < "B"
True
```

Simple Boolean expressions consist of the Boolean values `True` or `False`,

EIU
EASTERN INTERNATIONAL UNIVERSITY

if and if-else Statements

if-else Statements

The **if-else statement** is the most common type of selection statement. It is also called a **two-way selection statement**, because it directs the computer to make a choice between two alternative courses of action.

```
import math
area = float(input("Enter the area: "))
if area > 0:
    radius = math's(area / math.pi)
    print("The radius is", radius)
else:
    print("Error: the area must be a positive number")
```

Here is the Python syntax for the **if-else** statement:

```
if <condition>:
    <sequence of statements-1>
else:
    <sequence of statements-2>
```

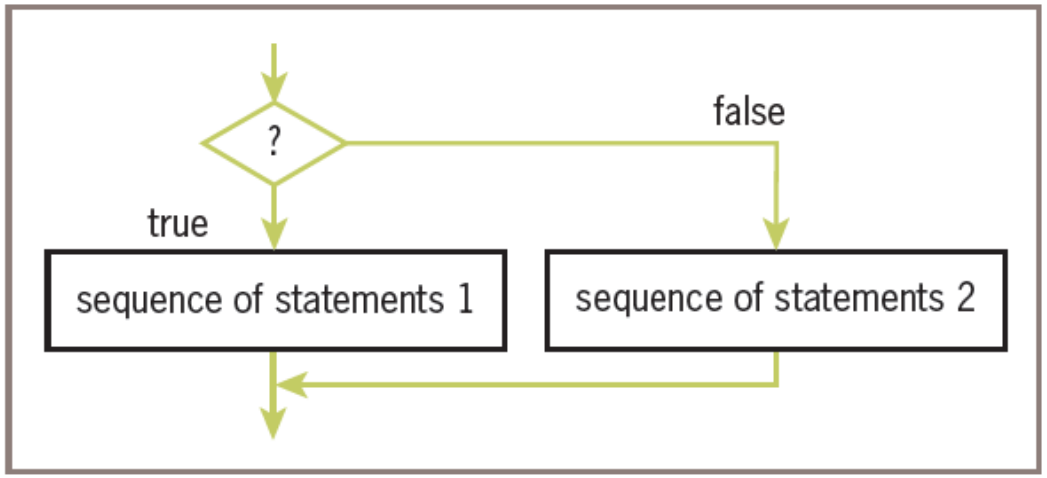


Figure 3-2 The semantics of the **if-else** statement

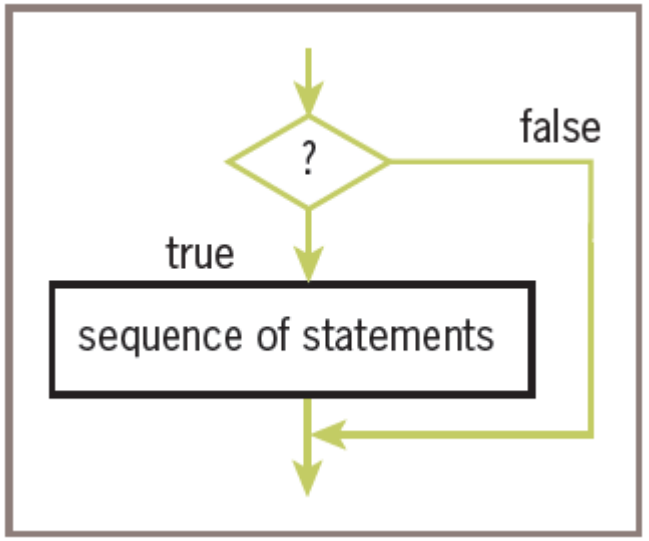
EIU
EASTERN INTERNATIONAL UNIVERSITY

```
first = int(input("Enter the first number: "))
second = int(input("Enter the second number: "))
if first > second:
    maximum = first
    minimum = second
else:
    maximum = second
    minimum = first
print("Maximum:", maximum)
print("Minimum:", minimum)
```

In the following example, the function **max** returns the largest of its arguments, whereas **min** returns the smallest of its arguments:

```
first = int(input("Enter the first number: "))
second = int(input("Enter the second number: "))
print("Maximum:", max(first, second))
print("Minimum:", min(first, second))
```

Figure 3-3
The semantics of the **if** statement



One-Way Selection Statements

```
if <condition>:
    <sequence of statements>
```

```
>>> if x < 0:
        x = -x
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Multi-Way `if` Statements

```
if <condition-1>:
    <sequence of statements-1>
elif <condition-n>:
    <sequence of statements-n>
else:
    <default sequence of statements>
```

```
number = int(input("Enter the numeric grade: "))
if number > 89:
    letter = 'A'
elif number > 79:
    letter = 'B'
elif number > 69:
    letter = 'C'
else:
    letter = 'F'
print("The letter grade is", letter)
```

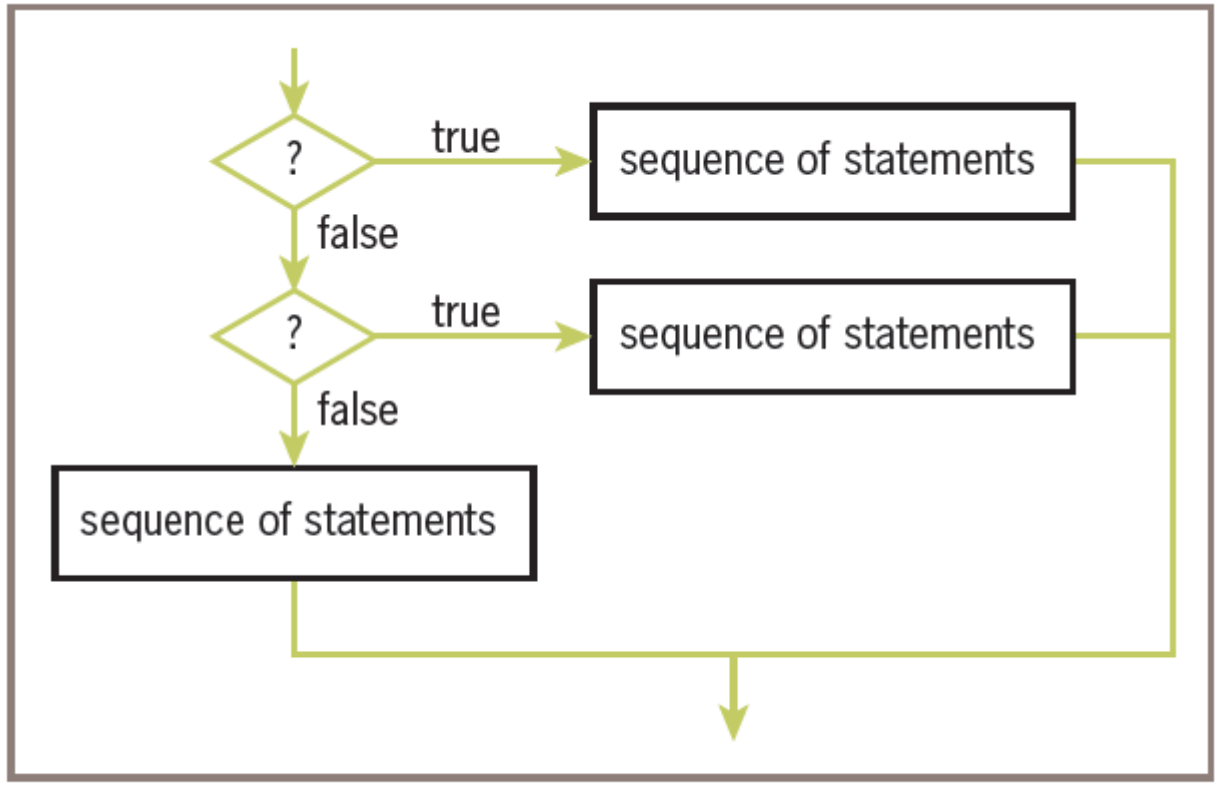


Figure 3-4 The semantics of the multi-way `if` statement

Logical Operators and Compound Boolean Expressions

```
number = int(input("Enter the numeric grade: "))
if number > 100:
    print("Error: grade must be between 100 and 0")
elif number < 0:
    print("Error: grade must be between 100 and 0")
else:
   # The code to compute and print the result goes here
```

Note that the first two conditions are associated with identical actions. Put another way, if either the first condition is true or the second condition is true, the program outputs the same error message.

The two conditions can be combined in a Boolean expression that uses the **logical operator or**. The resulting **compound Boolean expression** simplifies the code somewhat, as follows:

```
number = int(input("Enter the numeric grade: "))
if number > 100 or number < 0:
    print("Error: grade must be between 100 and 0")
else:
    # The code to compute and print the result goes here
```

```
number = int(input("Enter the numeric grade: "))
if number >= 0 and number <= 100:
    # The code to compute and print the result goes here
else:
    print("Error: grade must be between 100 and 0")
```

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

A	not A
True	False
False	True

Table 3-4 Operator precedence, from highest to lowest

Type of Operator	Operator Symbol
Exponentiation	**
Arithmetic negation	–
Multiplication, division, remainder	*, /, %
Addition, subtraction	+, –
Comparison	==, !=, <, >, <=, >=
Logical negation	**not**
Logical conjunction	**and**
Logical disjunction	**or**
Assignment	=

EIU
EASTERN INTERNATIONAL UNIVERSITY

Short-Circuit Evaluation

The Python virtual machine sometimes knows the value of a Boolean expression before it has evaluated all of its operands. For instance, in the expression `A and B`, if `A` is false, then so is the expression, and there is no need to evaluate `B`.

Likewise, in the expression `A or B`, if `A` is true, then so is the expression, and again there is no need to evaluate `B`. This approach, in which evaluation stops as soon as possible, is called **short-circuit evaluation**.

There are times when short-circuit evaluation is advantageous.

```
count = int(input("Enter the count: "))
theSum = int(input("Enter the sum: "))
if count > 0 and theSum // count > 10:
  print("average > 10")
else:
  print("count = 0 or average <= 10")
```

If the user enters 0 for the count, the condition contains a potential division by zero; however, because of short-circuit evaluation the division by zero is avoided.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Assume that **x** refers to a number. Write a code segment that prints the number's absolute value without using Python's **abs** function.
2. Write a loop that counts the number of space characters in a string. Recall that the space character is represented as `' '`.
3. Assume that the variables **x** and **y** refer to strings. Write a code segment that prints these strings in alphabetical order. You should assume that they are not equal.
4. Explain how to check for an invalid input number and prevent it being used in a program. You may assume that the user enters a number.
5. The variables **x** and **y** refer to numbers. Write a code segment that prompts the user for an arithmetic operator and prints the value obtained by applying that operator to **x** and **y**.

The while Loop

Conditional iteration requires that a condition be tested within the loop to determine whether the loop should continue. Such a condition is called the loop's **continuation condition**. If the continuation condition is false, the loop ends. If the continuation condition is true, the statements within the loop are executed again. The **while loop** is tailor-made for this type of control logic. Here is its syntax:

```
while <condition>:
    <sequence of statements>
```

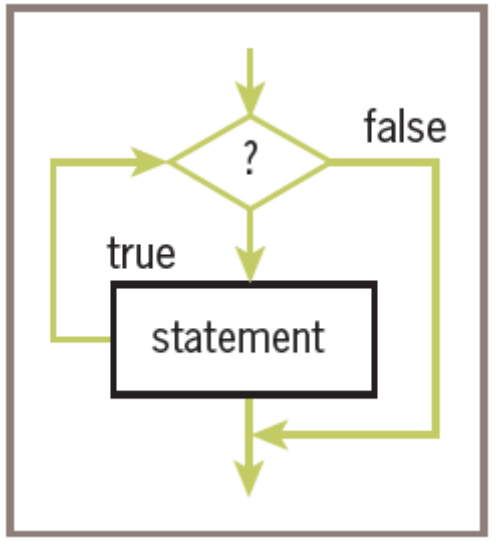


Figure 3-6 The semantics of a **while** loop

```
theSum = 0.0
data = input("Enter a number or just enter to quit: ")
while data != "":
    number = float(data)
    theSum += number
    data = input("Enter a number or just enter to quit: ")
print("The sum is", theSum)

Enter a number or just enter to quit: 3
Enter a number or just enter to quit: 4
Enter a number or just enter to quit: 5
Enter a number or just enter to quit:
The sum is 12.0
```

```
set the sum to 0.0
input a string
while the string is not the empty string
    convert the string to a float
    add the float to the sum
    input a string
print the sum
```

Count Control with a `while` Loop

```
# Summation with a for loop
theSum = 0
for count in range(1, 100001):
    theSum += count
print(theSum)

# Summation with a while loop
theSum = 0
count = 1
while count <= 100000:
    theSum += count
    count += 1
print(theSum)
```

```
# Counting down with a for loop
for count in range(10, 0, -1):
    print(count, end = " ")

# Counting down with a while loop
count = 10
while count >= 1:
    print(count, end = " ")
    count -= 1
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

The `while True` Loop and the `break` Statement

```
theSum = 0.0
while True:
    data = input("Enter a number
           or just enter to quit: ")
    if data == "":
        break
    number = float(data)
    theSum += number
print("The sum is", theSum)
```

```
done = False
while not done:
    number = int(input("Enter the numeric grade: "))
    if number >= 0 and number <= 100:
        done = True
    else:
        print("Error: grade must be between 100 and 0")
print(number)  # Just echo the valid input
```

```
while True:
    number = int(input("Enter the numeric grade: "))
    if number >= 0 and number <= 100:
        break
    else:
        print("Error: grade must be between 100 and 0")
print(number)  # Just echo the valid input
```

A trial run with just this segment shows the following interaction:

```
Enter the numeric grade: 101
Error: grade must be between 100 and 0
Enter the numeric grade: -1
Error: grade must be between 100 and 0
Enter the numeric grade: 45
45
```

Random Numbers

```
>>> import random
>>> for roll in range(10):
        print(random.randint(1, 6), end = " ")
2 4 6 4 3 2 3 6 2 2
```

```
import random
smaller = int(input("Enter the smaller number: "))
larger = int(input("Enter the larger number: "))
myNumber = random.randint(smaller, larger)
count = 0
while True:
    count += 1
    userNumber = int(input("Enter your guess: "))
    if userNumber < myNumber:
        print("Too small!")
    elif userNumber > myNumber:
        print("Too large!")
    else:
        print("Congratulations! You've got it in", count,
              "tries!")
        break
```

```
Enter the smaller number: 1
Enter the larger number: 100
Enter your guess: 50
Too small!
Enter your guess: 75
Too large!
Enter your guess: 63
Too small!
Enter your guess: 69
Too large!
Enter your guess: 66
Too large
Enter your guess: 65
You've got it in 6 tries!
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Translate the following **for** loops to equivalent **while** loops:

   a.
   ```
   for count in range(100):
       print(count)
   ```

   b.
   ```
   for count in range(1, 101):
       print(count)
   ```

   c.
   ```
   for count in range(100, 0, -1):
       print(count)
   ```

2. The factorial of an integer N is the product of the integers between 1 and N, inclusive. Write a **while** loop that computes the factorial of a given integer N.

3. The $\log_2$ of a given number N is given by M in the equation $N = 2^M$. Using integer arithmetic, the value of M is approximately equal to the number of times N can be evenly divided by 2 until it becomes 0. Write a loop that computes this approximation of the $\log_2$ of a given number N. You can check your code by importing the **math.log** function and evaluating the expression **round(math.log(N, 2))** (note that the **math.log** function returns a floating-point value).

EIU
EASTERN INTERNATIONAL UNIVERSITY

Projects

1. Write a program that accepts the lengths of three sides of a triangle as inputs. The program output should indicate whether or not the triangle is an equilateral triangle.

2. Write a program that accepts the lengths of three sides of a triangle as inputs. The program output should indicate whether or not the triangle is a right triangle. Recall from the Pythagorean theorem that in a right triangle, the square of one side equals the sum of the squares of the other two sides.

3. A local biologist needs a program to predict population growth. The inputs would be the initial number of organisms, the rate of growth (a real number greater than 0), the number of hours it takes to achieve this rate, and a number of hours during which the population grows. For example, one might start with a population of 500 organisms, a growth rate of 2, and a growth period to achieve this rate of 6 hours. Assuming that none of the organisms die, this would imply that this population would double in size every 6 hours. Thus, after allowing 6 hours for growth, we would have 1000 organisms, and after 12 hours, we would have 2000 organisms. Write a program that takes these inputs and displays a prediction of the total population.



EIU
EASTERN INTERNATIONAL UNIVERSITY

Projects



4. The German mathematician Gottfried Leibniz developed the following method to approximate the value of π:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \ldots$$

Write a program that allows the user to specify the number of iterations used in this approximation and that displays the resulting value.

5. Teachers in most school districts are paid on a schedule that provides a salary based on their number of years of teaching experience. For example, a beginning teacher in the Lexington School District might be paid $30,000 the first year. For each year of experience after this first year, up to 10 years, the teacher receives a 2% increase over the preceding value. Write a program that displays a salary schedule, in tabular format, for teachers in a school district. The inputs are the starting salary, the percentage increase, and the number of years in the schedule. Each row in the schedule should contain the year number and the salary for that year.

6. Write a program that receives a series of numbers from the user and allows the user to press the enter key to indicate that he or she is finished providing inputs. After the user presses the enter key, the program should print the sum of the numbers and their average.

EIU
EASTERN INTERNATIONAL UNIVERSITY

[CHAPTER] 4 Strings and Text Files

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

Accessing Characters and Substrings in Strings

The Structure of Strings

Unlike an integer, which cannot be decomposed into more primitive parts, a string is a **data structure**. A data structure is a compound unit that consists of several other pieces of data. A string is a sequence of zero or more characters. Recall that you can mention a Python string using either single quote marks or double quote marks. Here are some examples:

```
>>> "Hi there!"
'Hi there!'
>>> ""
''
>>> 'R'
'R'
```

```
>>> len("Hi there!")
9
>>> len("")
0
```

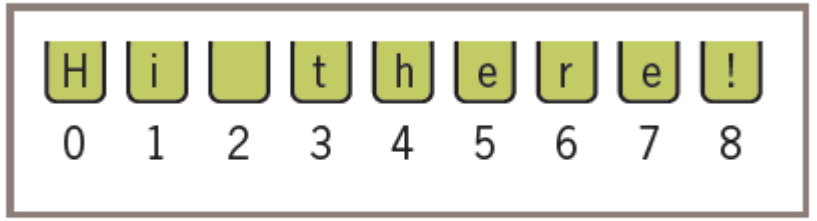


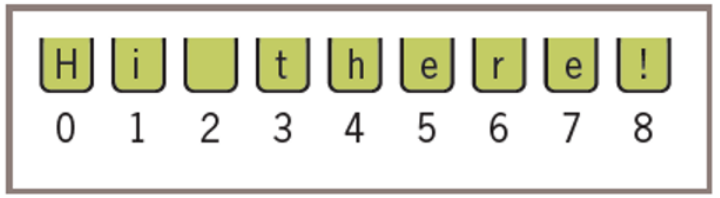
Figure 4-1 Characters and their positions in a string

EIU
EASTERN INTERNATIONAL UNIVERSITY

The Subscript Operator

<a string>[<an integer expression>]

```
>>> name = "Alan Turing"
>>> name[0]                 # Examine the first character
'A'
>>> name[3]                 # Examine the fourth character
'n'
>>> name[len(name)]         # Oops! An index error!
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>> name[len(name) - 1]     # Examine the last character
'g'
>>> name[-1]                # Shorthand for the last character
'g'
>>> name[-2]                # Shorthand for next to last character
'n'
```




```
>>> data = "Hi there!"
>>> for index in range(len(data)):
        print(index, data[index])

0 H
1 i
2
3 t
4 h
5 e
6 r
7 e
8 !
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Slicing for Substrings

You can use Python's subscript operator to obtain a substring through a process called **slicing**. To extract a substring, the programmer places a colon (:) in the subscript. An integer value can appear on either side of the colon.

Here are some examples that show how slicing is used:

```
>>> name = "myfile.txt"    # The entire string
>>> name[0:]
'myfile.txt'
>>> name[0:1]              # The first character
'm'
>>> name[0:2]              # The first two characters
'my'
>>> name[:len(name)]       # The entire string
'myfile.txt'
>>> name[-3:]              # The last three characters
'txt'
>>> name[2:6]              # Drill to extract 'file'
'file'
```

```
>>> fileList = ["myfile.txt", "myprogram.exe", "yourfile.txt"]
>>> for fileName in fileList:
        if ".txt" in fileName:
            print(fileName)
myfile.txt
yourfile.txt
```

Testing for a Substring with the `in` Operator

Exercises



1. Assume that the variable **`data`** refers to the string **`"myprogram.exe"`**. Write the values of the following expressions:
   a. **`data[2]`**
   b. **`data[-1]`**
   c. **`len(data)`**
   d. **`data[0:8]`**
2. Assume that the variable **`data`** refers to the string **`"myprogram.exe"`**. Write the expressions that perform the following tasks:
   a. Extract the substring **`"gram"`** from **`data`**.
   b. Truncate the extension **`".exe"`** from **`data`**.
   c. Extract the character at the middle position from **`data`**.
3. Assume that the variable **`myString`** refers to a string. Write a code segment that uses a loop to print the characters of the string in reverse order.
4. Assume that the variable **`myString`** refers to a string, and the variable **`reversedString`** refers to an empty string. Write a loop that adds the characters from **`myString`** to **`reversedString`** in reverse order.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Data Encryption

Encryption techniques are as old as the practice of sending and receiving messages. The sender **encrypts** a message by translating it to a secret code, called a **cipher text**. At the other end, the receiver **decrypts** the cipher text back to its original **plaintext** form. Both parties to this transaction must have at their disposal one or more **keys** that allow them to encrypt and decrypt messages.

A very simple encryption method that has been in use for thousands of years is called a **Caesar cipher**. Recall that the character set for text is ordered as a sequence of distinct values. This encryption strategy replaces each character in the plaintext with the character that occurs a given distance away in the sequence. For positive distances, the method wraps around to the beginning of the sequence to locate the replacement characters for those characters near its end. For example, if the distance value of a Caesar cipher equals three characters, the string `"invaders"` would be encrypted as `"lqydghuv"`. To decrypt this cipher text back to plaintext, you apply a method that uses the same distance value but looks to the left of each character for its replacement. This decryption method wraps around to the end of the sequence to find a replacement character for one near its beginning.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Figure 4-2 A Caesar cipher with distance +3 for the lowercase alphabet

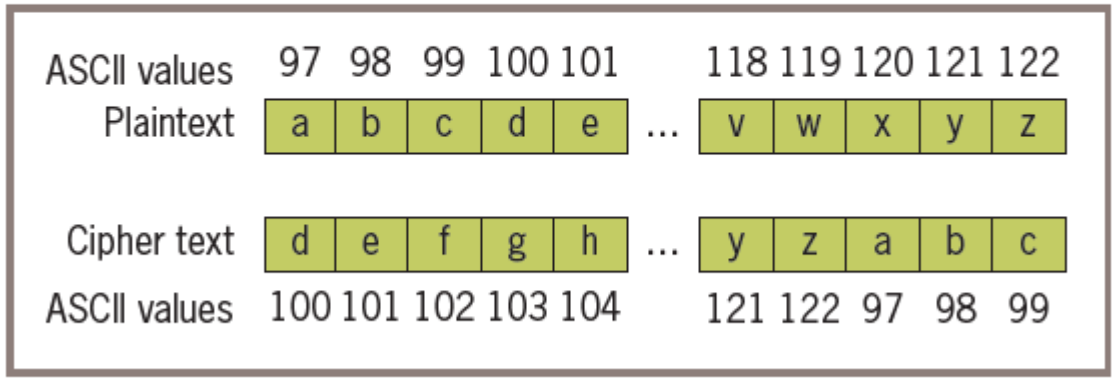


Figure 4-2 shows the first five and the last five plaintext characters of the lowercase alphabet and the corresponding cipher text characters, for a Caesar cipher with a distance of +3. The numeric ASCII values are listed above and below the characters.

Note the wraparound effect for the last three plaintext characters, whose cipher text characters start at the beginning of the alphabet. For example, the plaintext character 'x' with ASCII 120 maps to the cipher character 'a' with ASCII 97, because ASCII 120 is less than 3 characters from the end of the plaintext sequence.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Recall that the **ord** function returns the ordinal position of a character value in the ASCII sequence, whereas **chr** is the inverse function.

```
"""
File: encrypt.py
Encrypts an input string of lowercase letters and prints
the result. The other input is the distance value.
"""

plainText = input("Enter a one-word, lowercase message: ")
distance = int(input("Enter the distance value: "))
code = ""
for ch in plainText:
    ordvalue = ord(ch)
    cipherValue = ordvalue + distance
    if cipherValue > ord('z'):
        cipherValue = ord('a') + distance - \
                      (ord('z') - ordvalue + 1)
    code += chr(cipherValue)
print(code)
```

```
"""
File: decrypt.py
Decrypts an input string of lowercase letters and prints
the result. The other input is the distance value.
"""

code = input("Enter the coded text: ")
distance = int(input("Enter the distance value: "))
plainText = ""
for ch in code:
    ordvalue = ord(ch)
    cipherValue = ordvalue - distance
    if cipherValue < ord('a'):
        cipherValue = ord('z') - \
                      (distance - (ord('a') - ordvalue - 1))
    plainText += chr(cipherValue)
print(plainText)
```

Here are some executions of the two scripts in the IDLE shell:

```
Enter a one-word, lowercase message: invaders
Enter the distance value: 3
Lqydghuv
```

```
Enter the coded text: lqydghuv
Enter the distance value: 3
invaders
```

A more sophisticated encryption scheme is called a **block cipher**. A block cipher uses plaintext characters to compute two or more encrypted characters. This is accomplished by using a mathematical structure known as an **invertible matrix** to determine the values of the encrypted characters. The matrix provides the key in this method. The receiver uses the same matrix to decrypt the cipher text.



Exercises

1. Write the encrypted text of each of the following words using a Caesar cipher with a distance value of 3:
   a. python
   b. hacker
   c. wow

Strings and Number Systems

$$415_{10} = 4 * 10^2 + 1 * 10^1 + 5 * 10^0$$
$$= 4 * 100 + 1 * 10 + 5 * 1$$
$$= 400 + 10 + 5$$
$$= 415$$

415 in binary notation	110011111_2
415 in octal notation	637_8
415 in decimal notation	415_{10}
415 in hexadecimal notation	$19F_{16}$

$$1100111_2 = 1 * 2^6 + 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$$
$$= 1 * 64 + 1 * 32 + 0 * 16 + 0 * 8 + 1 * 4 + 1 * 2 + 1 * 1$$
$$= 64 + 32 + 4 + 2 + 1$$
$$= 103$$

```
Positional values  100  10  1
Positions            2   1  0
```

Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000

Converting Binary to Decimal

Converting Decimal to Binary

```
"""
File: binarytodecimal.py
Converts a string of bits to a decimal integer.
"""

bitString = input("Enter a string of bits: ")
decimal = 0
exponent = len(bitString) - 1
for digit in bitString:
    decimal = decimal + int(digit) * 2 ** exponent
    exponent = exponent - 1
print("The integer value is", decimal)
```

```
Enter a string of bits: 1111
The integer value is 15

Enter a string of bits: 101
The integer value is 5
```

```
"""
File: decimaltobinary.py
Converts a decimal integer to a string of bits.
"""

decimal = int(input("Enter a decimal integer: "))
if decimal == 0:
    print(0)
else:
    print("Quotient Remainder Binary")
    bitString = ""
    while decimal > 0:
        remainder = decimal % 2
        decimal = decimal // 2
        bitString = str(remainder) + bitString
        print("%5d%8d%12s" % (decimal, remainder,
                              bitString))
  print("The binary representation is", bitString)
```

```
Enter a decimal integer: 34
Quotient Remainder Binary
   17       0           0
    8       1          10
    4       0         010
    2       0        0010
    1       0       00010
    0       1      100010
The binary representation is 100010
```

Exercises



1. Translate each of the following numbers to decimal numbers:
   a. 11001_2
   b. 100000_2
   c. 11111_2
2. Translate each of the following numbers to binary numbers:
   a. 47_{10}
   b. 127_{10}
   c. 64_{10}
3. Translate each of the following numbers to binary numbers:
   a. 47_8
   b. 127_8
   c. 64_8
4. Translate each of the following numbers to decimal numbers:
   a. 47_8
   b. 127_8
   c. 64_8
5. Translate each of the following numbers to decimal numbers:
   a. 47_{16}
   b. 127_{16}
   c. AA_{16}

String Methods

<an object>.<method name>(<argument-1>,..., <argument-*n*>)

```
>>> sentence = input("Enter a sentence: ")
Enter a sentence: This sentence has no long words.
>>> listOfWords = sentence.split()
>>> print("There are", len(listOfWords), "words.")
There are 6 words.
>>> sum = 0
>>> for word in listOfWords:
        sum += len(word)
>>> print("The average word length is", sum / len(listOfWords))
The average word length is 4.5
```

```
>>> words = s.split()
>>> words
['Hi', 'there!']
>>> " ".join(words)
'Hithere!'
>>> " ". join(words)
'Hi there!'
```

```
>>> s.lower()
'hi there!'
>>> s.upper()
'HI THERE!'
>>> s.replace('i', 'o')
'Ho there!'
>>> " Hi there! ".strip()
'Hi there!'
```

```
>>> s = "Hi there!"
>>> len(s)
9
>>> s.center(11)
' Hi there! '
>>> s.count('e')
2
>>> s.endswith("there!")
True
>>> s.startswith("Hi")
True
>>> s.find("the")
3
>>> s.isalpha()
False
>>> 'abc'.isalpha()
True
>>> "326".isdigit()
True
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

String Method	What it Does
`s.center(width)`	Returns a copy of **s** centered within the given number of columns.
`s.count(sub [, start [, end]])`	Returns the number of non-overlapping occurrences of substring **sub** in **s**. Optional arguments **start** and **end** are interpreted as in slice notation.
`s.endswith(sub)`	Returns **True** if **s** ends with **sub** or **False** otherwise.
`s.find(sub [, start [, end]])`	Returns the lowest index in **s** where substring **sub** is found. Optional arguments **start** and **end** are interpreted as in slice notation.
`s.isalpha()`	Returns **True** if **s** contains only letters or **False** otherwise.
`s.isdigit()`	Returns **True** if **s** contains only digits or **False** otherwise.
`s.join(sequence)`	Returns a string that is the concatenation of the strings in the sequence. The separator between elements is **s**.

String Method	What it Does
`s.lower()`	Returns a copy of **s** converted to lowercase.
`s.replace(old, new [, count])`	Returns a copy of **s** with all occurrences of substring **old** replaced by **new**. If the optional argument **count** is given, only the first **count** occurrences are replaced.
`s.split([sep])`	Returns a list of the words in **s**, using **sep** as the delimiter string. If **sep** is not specified, any whitespace string is a separator.
`s. startswith(sub)`	Returns **True** if **s** starts with **sub** or **False** otherwise.
`s.strip([aString])`	Returns a copy of **s** with leading and trailing whitespace (tabs, spaces, newlines) removed. If **aString** is given, remove characters in **aString** instead.
`s.upper()`	Returns a copy of **s** converted to uppercase.

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> "myfile.txt".split('.')
['myfile', 'txt']
>>> "myfile.py".split('.')
['myfile', 'py']
>>> "myfile.html".split('.')
['myfile', 'html']
```

The method **split** returns a list of words in the string upon which it is called. This method assumes that the default separator character between the words is a space.

Note that the extension, regardless of its length, is the last string in each list. You can now use the subscript **[-1]**, which also extracts the last element in a list, to write a general expression for obtaining any filename's extension, as follows:

```
filename.split('.')[-1]
```


Exercises

1. Assume that the variable **data** refers to the string **"Python rules!"**. Use a string method from Table 4-2 to perform the following tasks:
   a. Obtain a list of the words in the string.
   b. Convert the string to uppercase.
   c. Locate the position of the string **"rules"**.
   d. Replace the exclamation point with a question mark.



Text Files

Writing Text to a File

Data can be output to a text file using a file object. Python's **open** function, which expects a file name and a **mode string** as arguments, opens a connection to the file on disk and returns a file object.

The mode string is `'r'` for input files and `'w'` for output files. Thus, the following code opens a file object on a file named **myfile.txt** for output:

```
>>> f = open("myfile.txt", 'w')
```

If the file does not exist, it is created with the given filename. If the file already exists, Python opens it. When an existing file is opened for output, any data already in it are erased.

String data are written (or output) to a file using the method **`write`** with the file object. The **`write`** method expects a single string argument. If you want the output text to end with a newline, you must include the escape character `'\n'` in the string.

```
>>> f.write("First line.\nSecond line.\n")
```

When all of the outputs are finished, the file should be closed using the method **`close`**, as follows:

```
>>> f.close()
```

Reading Text from a File

```
>>> f = open("myfile.txt", 'r')
```

You open a file for input in a similar manner to opening a file for output. The only thing that changes is the mode string, which, in the case of opening a file for input, is **'r'**. However, if a file with that name is not accessible, Python raises an error. Here is the code for opening **myfile.txt** for input:

```
>>> text = f.read()
>>> text
'First line.\nSecond line.\n'
>>> print(text)
First line.
Second line.
```

There are several ways to read data from an input file. The simplest way is to use the file method **read** to input the entire contents of the file as a single string. If the file contains multiple lines of text, the newline characters will be embedded in this string. The next session shows how to use the method **read**:

```
>>> f = open("myfile.txt", 'r')
>>> while True:
        line = f.readline()
        if line == "":
            break
        print(line)
First line.

Second line.
```

```
>>> f = open("myfile.txt", 'r')
>>> for line in f:
        print(line)
First line.

Second line.
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Writing Numbers to a File

The file method **write** expects a string as an argument. Therefore, other types of data, such as integers or floating-point numbers, must first be converted to strings before being written to an output file.

Five hundred random integers between 1 and 500 are generated and written to a text file named **integers.txt**. The newline character is the separator.

```
import random
f = open("integers.txt", 'w')
for count in range(500):
    number = random.randint(1, 500)
    f.write(str(number) + '\n')
f.close()
```

Reading Numbers from a File

```
f = open("integers.txt", 'r')
theSum = 0
for line in f:
    line = line.strip()
    number = int(line)
    theSum += number
print("The sum is", theSum)
```

```
f = open("integers.txt", 'r')
theSum = 0
for line in f:
    wordlist = line.split()
    for word in wordlist:
        number = int(word)
        theSum += number
print("The sum is", theSum)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Table 4-3 Some **file** operations

Method	What it Does
`open(filename, mode)`	Opens a file at the given filename and returns a file object. The `mode` can be `'r'`, `'w'`, `'rw'`, or `'a'`. The last two values, `'rw'` and `'a'`, mean read/write and append, respectively.
`f.close()`	Closes an output file. Not needed for input files.
`f.write(aString)`	Outputs `aString` to a file.
`f.read()`	Inputs the contents of a file and returns them as a single string. Returns "" if the end of file is reached.
`f.readline()`	Inputs a line of text and returns it as a string, including the newline. Returns "" if the end of file is reached.

Accessing and Manipulating Files and Directories on Disk

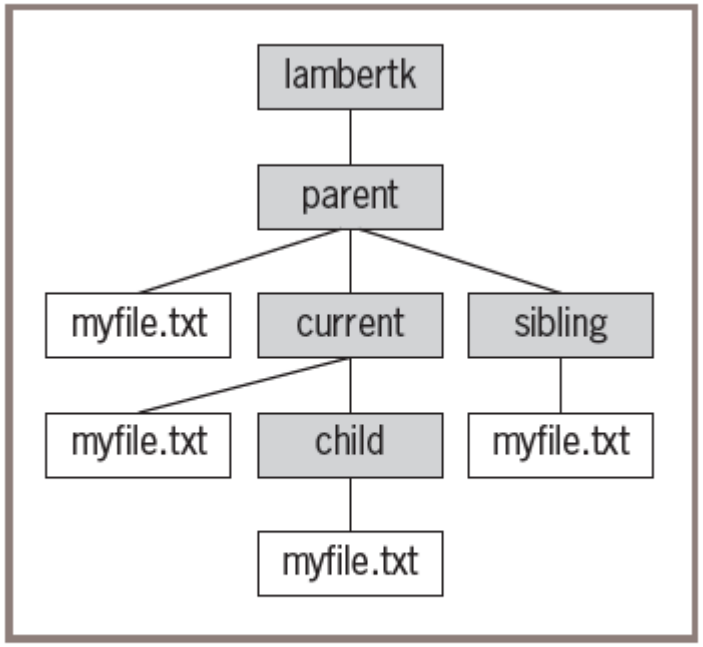


Figure 4-6 A portion of a file system

Table 4-4

Relative pathnames from **current** directory to **myfile.txt** in Figure 4-6

Pathname	Target Directory
myfile.txt	current
child/myfile.txt	child
../myfile.txt	parent
../sibling/myfile.txt	sibling

```
f = open("/Users/lambertk/parent/current/child/myfile.txt", 'r')

childFile = open("child/myfile.txt", 'r')
parentFile = open("../myfile.txt", 'r')
siblingFile = open("../sibling/myfile.txt", 'r')
```

Copyright © 2019 Cengage Learning. All Rights Reserved. May not be copied, scanned, or

EIU
EASTERN INTERNATIONAL UNIVERSITY

Table 4-5 Some file system functions

os Module Function	What it Does
`chdir(path)`	Changes the current working directory to **path**.
`getcwd()`	Returns the path of the current working directory.
`listdir(path)`	Returns a list of the names in directory named **path**.
`mkdir(path)`	Creates a new directory named **path** and places it in the current working directory.
`remove(path)`	Removes the file named **path** from the current working directory.
`rename(old, new)`	Renames the file or directory named **old** to **new**.
`rmdir(path)`	Removes the directory named **path** from the current working directory.
`sep`	A variable that holds the separator character (`'/'` or `'\'`) of the current file system.

```
import os
currentDirectoryPath = os.getcwd()
listOfFileNames = os.listdir(currentDirectoryPath)
for name in listofFileNames:
    if ".py" in name:
        print(name)
```

Table 4-6 More file system functions

os.path Module Function	What it Does
`exists(path)`	Returns **True** if **path** exists and **False** otherwise.
`isdir(path)`	Returns **True** if **path** names a directory and **False** otherwise.
`isfile(path)`	Returns **True** if **path** names a file and **False** otherwise.
`getsize(path)`	Returns the size of the object names by **path** in bytes.
`normcase(path)`	Converts path to a pathname appropriate for the current file system; for example, converts forward slashes to backslashes and letters to lowercase on a Windows system.

EIU
EASTERN INTERNATIONAL UNIVERSITY

`os.rename("oldname.txt", "newname.txt")`

is a function called on its defining module, not a method called on an object.

Exercises



1. Write a code segment that opens a file named **myfile.txt** for input and prints the number of lines in the file.
2. Write a code segment that opens a file for input and prints the number of four-letter words in the file.
3. Assume that a file contains integers separated by newlines. Write a code segment that opens the file and prints the average value of the integers.
4. Write a code segment that prints the names of all of the items in the current working directory.
5. Write a code segment that prompts the user for a filename. If the file exists, the program should print its contents on the terminal. Otherwise, it should print an error message.